

كلية الهندسة المعلوماتية
جامعة دمشق

مشروع مادة البرمجة المتوازية

محرك معالجة عالي الأداء لنظام تجارة الكترونية-

اعداد الطلاب:

محمد إسماعيل طه

محمد انس جاويش

عبدالرحمن الويش

عبدالرحيم نبيل المعاني

مقدمة:

في ظل النمو المتسارع للتجارة الإلكترونية، أصبحت الأنظمة البرمجية تواجه تحدياً كبيراً في الحفاظ على استقرارها وأدائها أثناء فترات الذروة وتدفق آلاف المستخدمين في نفس اللحظة لشراء المنتجات.

يقدم هذا المشروع نظاماً خلفياً متكاملًا لمتجر الكتروني تم بناؤه باستعمال الاطار العمل لارافل و لم يقتصر فيه تركيز على متطلبات الأساسية ، بل تم تصميمه كنموذج عملي للأنظمة ذات الأداء العالي ومواجهة المشاكل المعقدة في توازي البيانات

مهام النظام:

أولاً: نظام إدارة الهوية وتأمين الوصول

يهدف هذا النظام إلى حماية واجهات التطبيق والتحقق من هوية المستخدمين وصلاحياتهم باستخدام تقنية التوكنات الأمنية المعتمدة على مكتبة (Laravel Sanctum) وتتنوع مهامه كالتالي:

- **تسجيل الحسابات الجديدة** يستقبل بيانات العميل ويقوم بتشفير كلمة المرور فوراً لحمايتها :
دقاعدة البيانات، ثم يصدر توكن أمني فوري للمستخدم

تسجيل الدخول للنظام: يتحقق من مطابقة البريد الإلكتروني وكلمة المرور المسجلة، وفي حال النجاح يمنح العميل رمز صلاحية مشفر يسمى (Bearer Token) للوصول للخدمات المحمية.

: تسجيل الخروج الآمن

يقوم بحذف وإبطال رمز الصلاحية الحالي للمستخدم تماماً من قاعدة البيانات لمنع إعادة استخدامه من أي طرف آخر.

- **جلب بيانات المستخدم** يتيح للنظام التعرف الفوري على هوية الحساب الذي يقوم بالطلب حالياً وعرض بياناته الشخصية بأمان

ثانياً: نظام إدارة سلة المشتريات ديناميكياً

يتولى هذا القسم إدارة وتحديث المنتجات التي يختارها العميل قبل الشراء الفعلي، مع حساب الأسعار تلقائياً في الخلفية

- **يجلب السلة الحالية للمستخدم مع تفاصيل المنتجات، وتمت كتابة:** استعراض محتويات السلة
لحماية خادم قاعدة البيانات من استعلامات المنهكة (Eager Loading) الاستعلام برمجياً بطريقة الزائدة.

- يفحص السلة أولاً؛ إذا كان المنتج مضافاً مسبقاً يقوم : إضافة المنتجات للسلة: إضافة المنتجات للسلة .
بتحديث الكمية فقط، وإذا كان جديداً ينشئ عنصراً جديداً ويحسب السعر الفرعي له
- يتيح للعميل زيادة أو تقليل كمية أي منتج مع إعادة حساب المجموع : تعديل كميات العناصر .
الفرعي بناءً على سعر السلعة الأصلي
- يوفر خيارات لحذف منتج معين أو مسح السلة بالكامل وتصفير : حذف المنتجات وتفرغ السلة .
إجمالي السعر لتصبح جاهزة لعمليات تسوق جديدة

• ثالثاً: نظام معالجة الطلبات الآمن وعالي الأداء .

- يمثل هذا النظام العصب التقني للمشروع، حيث جرى تصميمه برمجياً للصدود أمام التدقق :
المتزامن الضخم للطلبات ومنع حدوث أي تضارب في المخزن
- يسمح لكل مستخدم بمراجعة سجل طلباته التاريخية أو تتبع طلب : استعراض ومتابعة الطلبات .
معين مع حظر الوصول لبيانات أي مستخدم آخر
- إلغاء الطلبات : يتيح تحديث حالة الطلب الحالي وتحويله إلى ملغي بناءً على رغبة العميل.
- يطبق خاصية القفل الفوري لقاعدة : منع البيع الوهمي : (Checkout) معالجة الشراء النهائي
على صف المنتج، مما يمنع شراء منتج نافذ من قبل أكثر من (Lock for Update) البيانات
..مستخدم في نفس اللحظة
- يتصل برمجياً ببوابات الدفع الخارجية، وفي حال فشل سحب الأموال، يتم : محاكاة بوابة الدفع
لإعادة المنتجات للمخزن فوراً وكأن (Database Transaction) تفعيل ميزة التراجع الكامل
شياً لم يكن
- يسجل الفاتورة في جداول المدفوعات ويحول حالة الطلب إلى مكتمل، ثم يصفر : تأكيد الحركة .
السلة

• رابعاً: نظام إدارة المنتجات والمخزون .

- يوفر هذا النظام الواجهات البرمجية الأساسية للتحكم بالسلع والخدمات المعروضة في المتجر :
الإلكتروني
- يجلب المنتجات من قاعدة البيانات باستخدام نظام الترميز والتجزئة : عرض المنتجات بأداء مُحسن
لضمان سرعة تحميل البيانات واختصار حجم الردود
- يتيح عمليات الإضافة والتحديث والحذف للمنتجات، مع معالجة البيانات والتحقق : التحكم بالبيانات
من صحة المدخلات عبر كلاسات منفصلة لحماية السيرفر

خامساً: نظام الجدولة والمهام الخلفية المنفصلة .

- يعمل هذا النظام على تحسين تجربة المستخدم وتقليل وقت استجابة السيرفر عبر عزل المهام (Queues) الثقيلة ومعالجتها في طابور خلفي مستقل
- مهمة خلفية معزولة تقوم بتحويل بيانات الشراء وقالب العرض إلى ملف: توليد الفواتير تلقائياً .
رسمي، ثم تخزينه في خادم الملفات المستقل (PDF)
- يتولى بناء وإرسال بريد إلكتروني رسمي للعميل فور نجاح عملية :إرسال بريد التأكيد الفوري .
الدفع، دون إجبار العميل على الانتظار على الشاشة حتى ينتهي الإرسال

متطلبات غير وظيفية:

حماية البيانات المشتركة من التضارب:

توصيف مشكلة حالة السباق:

تحدث هذه المشكلة البرمجية عند تدفق كمية ضخمة من الطلبات المتزامنة لقراءة وتعديل نفس السجل الخاص بكمية المنتج في قاعدة البيانات وفي نفس الجزء من الثانية. وبسبب غياب العزل الصارم، تقرأ تلك الطلبات المتزامنة قيمة المخزن القديمة معاً، مما يتسبب في معضلة البيع الوهمي الزائد عن الحد الفعلي

: البيئة التجريبية وأثر الخادم المحلي:

تمت محاكاة المشكلة عبر أداة فحص الجهد من خلال إرسال عشرين طلباً متزامناً لشراء منتج يتوفر منه ..خمس قطع فقط داخل المخزن

إثبات الثغرة قبل التحسين البرمجي

نجحت الطلبات 19 الأولى فقط لأنها استهلكت المخزن المتاح، وتظهر في الأداة باللون الأخضر

عند وصول الطلب 20 ومحاولة خصم كمية إضافية تحت الصفر، انهيار استعمال قاعدة البيانات تماماً ورمى السيرفر خطأ داخلي من فئة 500، لأن حقل الكمية معرف في الجداول بأنه لا يقبل القيم السالبة

: المعالجة وسلوك النظام بعد التحسين

تم حل المشكلة جذرياً من خلال تطبيق تقنية القفل الصارم على مستوى السطر داخل معاملات قاعدة البيانات، وذلك باستخدام أمر القفل المخصص للتحديث.

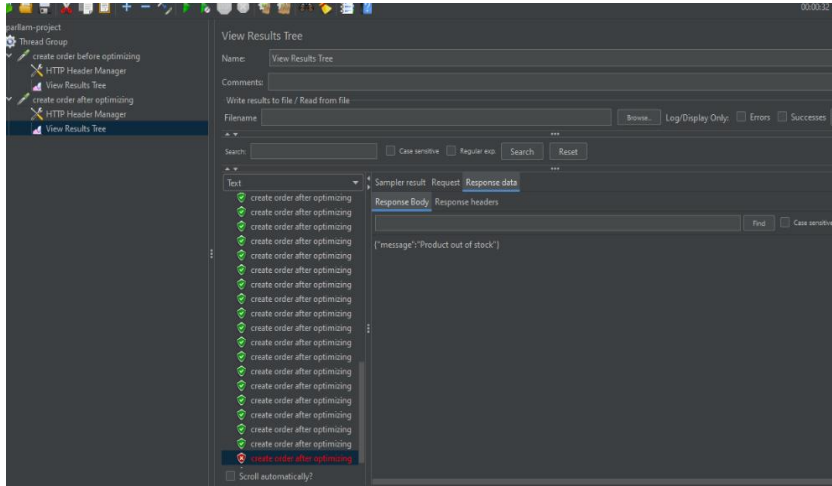
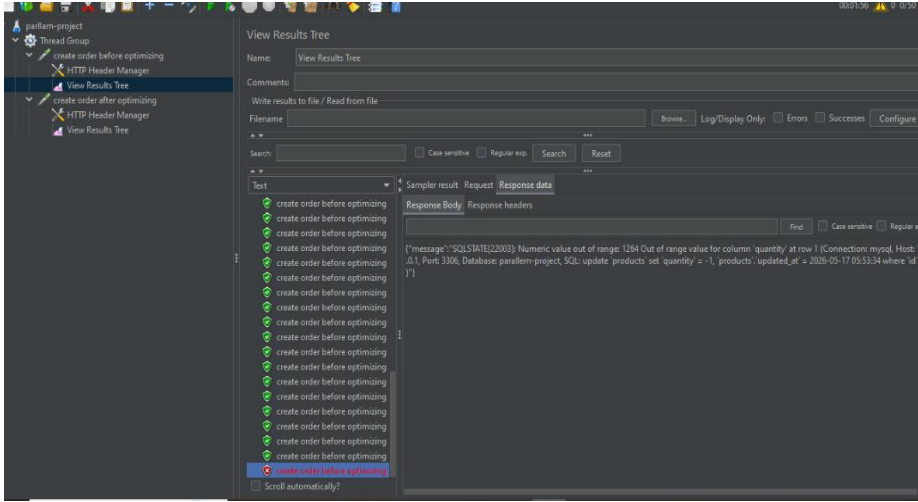
الطلب الأول المتدفق يقوم بقفل سطر المنتج فوراً في قاعدة البيانات، مما يجبر بقية الطلبات المتزامنة على الانتظار في طابور آمن حتى تنتهي العملية الأولى

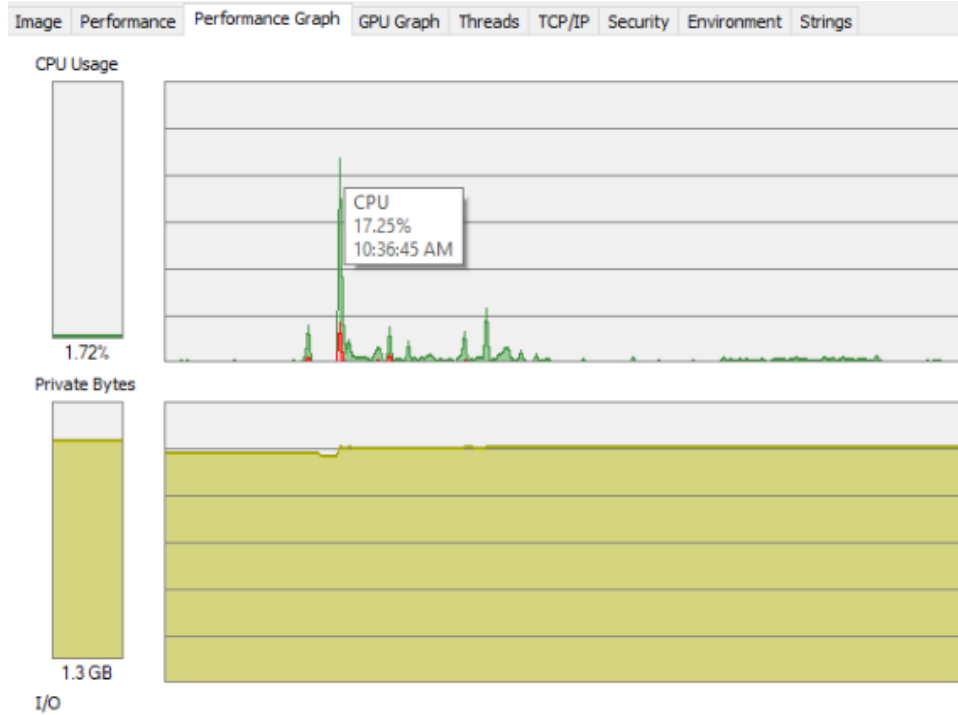
- اتمت الطلبات 19 الأولى بنجاح وتستهلك المخزن، وعندما يفتح القفل ويأتي الدور على الطلب 20، يجد أن المخزن أصبح

صفراً، فيقوم الكود برفضه برمجياً بشكل ذكي ويعيد رسالة مخصصة ونظيفة للمستخدم تفيد بنفاذ الكمية، وتظهر الطلبات المتبقية باللون الأحمر دون حدوث أي انهيار أو أخطاء في السيرفر

إدارة الموارد الحاسوبية:

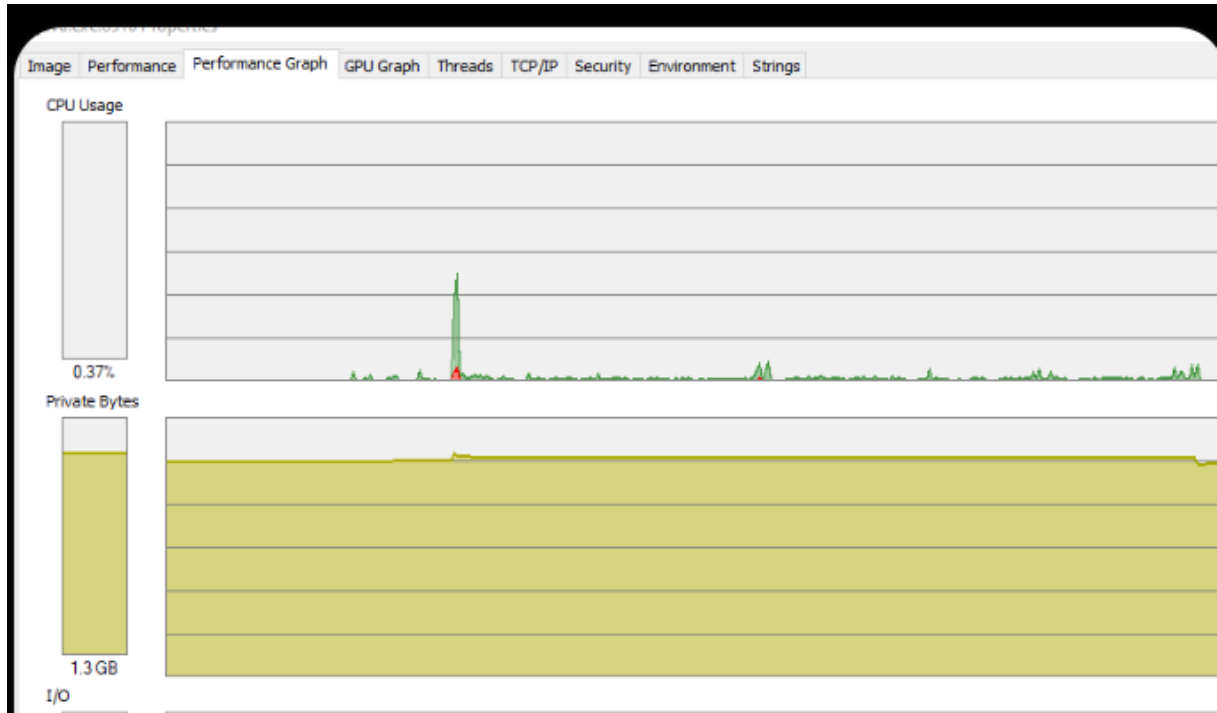
عند إرسال 200 طلب متزامن في نفس الملي ثانية لاختبار دالة الشراء يظهر الرسم البياني ضغطاً إلى 17.25% مع تذبذب حاد وارتفاع ملحوظ في CPU مفاجئاً على المعالج حيث قفز استهلاك الـ وتعود هذه القفزة البرمجية العشوائية إلى حدوث حالة السباق أو الـ (I/O) عمليات القراءة والكتابة حيث حاولت الطلبات المتداخلة تعديل كميات المنتجات في نفس اللحظة مما Race Condition تسبب في تكرار العمليات وقبول طلبات مبيعات زائدة تتجاوز المخزون الفعلي مع استهلاك طاقة المعالج في محاولات معالجة البيانات المتعارضة





بعد التحسين:

عند إعادة نفس الاختبار وإرسال 200 طلب متزامن بعد تطبيق ميزة lockForUpdate الـ يظهر الرسم البياني الثاني استقراراً تاماً وانخفاضاً حاداً في استهلاك المعالج ليثبت عند 0.37% فقط مع اختفاء التذبذبات العشوائية وهبوط مؤشر الـ I/O إلى مستويات دنيا وتؤكد هذه النتيجة نجاح الحل البرمجي في تنظيم تدفق الطلبات المتزامنة وجعلها تنتظر دورها في الطابور بشكل متسلسل مما منع تكرار العمليات وحسم مسألة الـ Race Condition تماماً وحافظ على ثبات السيرفر وسلامة بيانات المخزن واستطعنا الحصول على هذا التحسن باستخدام طريقتان عمل قمنا بهم (Funnel) حيث تسمح لنا بتحديد عدد العمليات المتزامنة التي سيستطيع السيرفر التعامل معها في نفس اللحظة الزمنية وأيضا أضفنا طبقة ثانية وهي (Global Rate Limiter) للتحكم في عدد الطلبات لاجمالي التي ستاتي الى السيرفر وربطها اما باستخدام key او باستخدام ip



معالجة الغير متزامنة:

تسبب معالجة البيانات الإضافية كإرسال الإيميل و إنشاء فواتير دفع الى استهلاكاً مفرطاً للذاكرة وبطء استجابة الموقع مما قد يؤدي لانتهياره وتجربة مستخدم سيئة لذلك يعمل الطابور

على عزل المهام الثقيلة وتأجيل تنفيذها في الخلفية بشكل غير متزامن لتوفير رد فوري للمتصفح

عداد جداول قاعدة البيانات والتميز بين الطلب غير المدفوع والطلب المؤكد لبدء المعالجة

إنشاء بوابة دفع وهمية لاختبار تحول حالات الطلبات وتغذية النظام بالبيانات في بيئة التطوير

تأمين عملية الشراء الفورية عبر ميزة حظر التحديث المتزامن لحماية المخزون من البيع الزائد

تشغيل مهمة توليد الفواتير بصيغة GenerateInvoicePdf عبر طابور الـ Queue لتنفيذها في الخلفية دون إشغال المتصفح

تشغيل مهمة إرسال بريد تأكيد الشراء SendOrderConfirmationEmail عبر طابور الـ Queue لتتصل بخوادم البريد الخارجي بشكل منفصل

إطلاق المهام الخلفية للفواتير والرسائل عبر الـ Queue فور تأكيد الطلب مباشرة لضمان سرعة تصفح الموقع

بناء مهمة معالجة مبيعات اليوم على دفعات مقسمة تشمل كل دفعة مئة طلب مع حمايتها بنظام العمليات المترابطة

ربط مهمة الدفعات بالمجدول الزمني لتعمل تلقائياً كل دقيقة مع تفعيل خاصية منع تداخل المهام
تشغيل المجدول ومعالج المهام عبر سطر الأوامر وبناء واجهة عرض برمجية لمراقبة حركات المخزن بشكل فوري ومباشر

معالجة البيانات الضخمة على دفعات:

في الأنظمة البرمجية الضخمة ومشاريع التجارة الإلكترونية تسبب معالجة آلاف السجلات دفعة واحدة مثل المبيعات اليومية ضغطاً هائلاً على ذاكرة السيرفر العشوائية مما يؤدي إلى بطء استجابة الموقع وربما انهياره

تكمّن الفائدة الجوهرية لاستخدام تقنية الدفعات أو الـ Chunks في حماية سيرفر المشروع عن طريق تقسيم الكتلة البيانات الضخمة إلى مجموعات صغيرة محددة العدد حيث يتم جلب الدفعة الأولى ومعالجتها ثم تفرغها من الذاكرة تماماً قبل الانتقال للدفعة التالية مما يضمن ثبات استهلاك الذاكرة العشوائية عند حد منخفض جداً مهما تضاعف حجم المبيعات ويمنع انهيار النظام ويوفر استعلامات خارقة السرعة تعتمد على فهارس قاعدة البيانات

خطوات العمل :

- إعداد البنية التحتية لقاعدة البيانات عبر تجهيز جدول الطلبات الأساسي وجدول حركات المخزن المخصص للأرشفة والتتبع
- بناء المهمة الخلفية وتطبيق تقنية معالجة الدفعات الذكية بداخلها لتقوم بجلب الطلبات المؤكدة لليوم الحالي فقط على مجموعات مقسمة تشمل كل مجموعة مئة طلب

```
public function handle()
{
    Order::where('status', 'confirmed')
        ->whereDate('created_at', today())
        ->chunkById(100, function ($orders) {
            DB::transaction(function () use ($orders) {
                foreach ($orders as $order) {
                    InventoryLog::create([
                        'product_id' => $order->product_id,
                        'previous_quantity' => isset($order->quantity_before_order) ? $order->quantity_before_order : 100,
                        'new_quantity' => isset($order->quantity_after_order) ? $order->quantity_after_order : 90,
                        'operation_type' => 'Batch Processing (Daily Sales Job)',
                        'created_by' => isset($order->user_id) ? $order->user_id : 1,
                    ]);
                }
            });
        });
}
```


- دمج نظام العمليات الآمنة لقاعدة البيانات داخل المهمة الخلفية لضمان تراجع النظام عن الدفعة كاملة في حال حدوث أي خطأ طارئ منعاً لتلف البيانات
- ربط المهمة الخلفية مع المجدول الزمني الخاص بإطار العمل لتعمل تلقائياً عند بداية كل دقيقة
- تشغيل معالج المهام والمجدول عبر سطر الأوامر في بيئة التطوير لمراقبة التقاط السجلات وتدفق العمليات دقيقة بدقيقة في الخلفية

```
• Schedule::job(new ProcessDailySalesReport)->everyMinute();
```

- بناء واجهة عرض برمجية بسيطة مرتبطة بقاعدة البيانات تتيح للإدارة مراقبة حركات المخزن والكميات السابقة والجديدة بشكل مرئي ومباشر وفوري

- توزيع الاحمال :

لمواجهة تدفق الطلبات الضخمة وحماية المتجر من الانهيار، تم استخدام سيرفر Nginx كموازن للأحمال في الواجهة الأمامية للنظام. يتلقى Nginx كافة طلبات المستخدمين ويقوم بتوزيعها بشكل ذكي وعادل على ثلاث نسخ حية ومستقلة من تطبيق لارافل تعمل بالتوازي في الخلفية.

عند حدوث ضغط شراء متزامن، تجري العملية كالتالي:

- **نقطة وصول موحدة:** يستقبل Nginx الطلبات عبر المنفذ الرئيسي 9090

```
}
```

- **التدوير الآلي للطلبات:** يقوم Nginx بتمرير الطلبات وتدويرها

```
http {
```

تلقائياً بين النسخ الثلاث؛ حيث يعالج الخادم الأول الطلب الأول، بينما يتجه الطلب الثاني فوراً للخادم الثاني، وهكذا.

```
upstream laravel_backend {
    server 127.0.0.1:8000;
    server 127.0.0.1:8001;
    server 127.0.0.1:8002;
}
```

- **تقسيم الجهد:** هذا التوزيع يضمن خفض استهلاك المعالج والذاكرة لكل خادم إلى الثلث، مما يسرع وقت الاستجابة للمستخدمين.

- **استمرارية الخدمة:** في حال تعرض أحد الخوادم لضغط شديد

أو توقف مفاجئ، يتجاهله Nginx تلقائياً ويوجه حركة المرور للخوادم الأخرى المتاحة، مما يضمن بقاء المتجر متصلاً دون أي انقطاع.